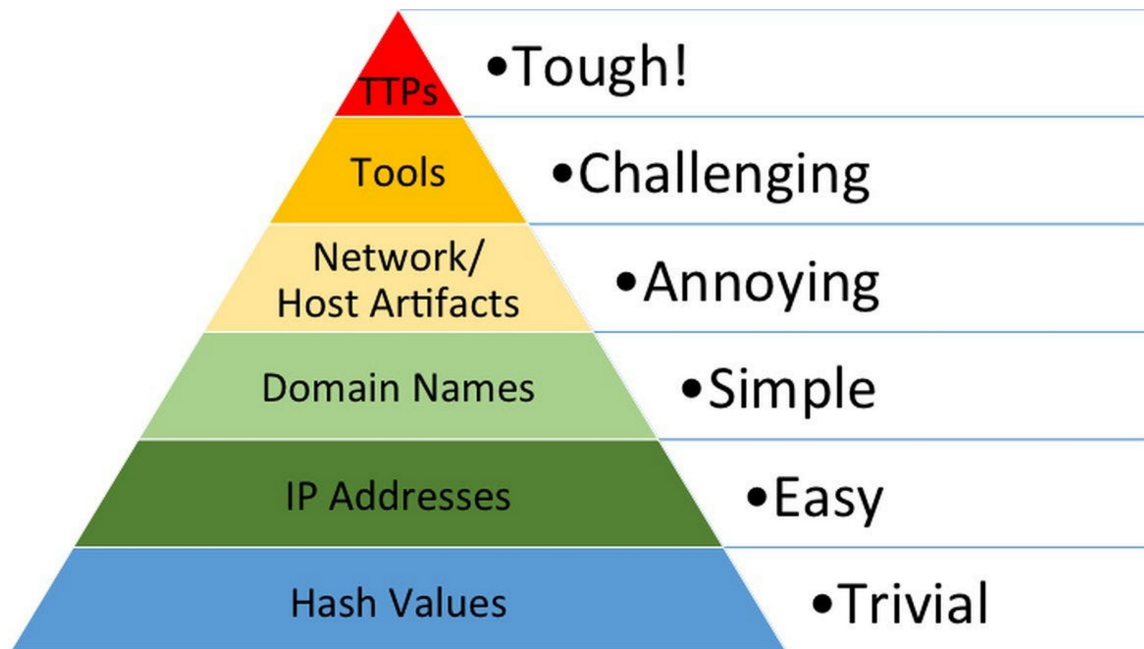




What is the Pyramid of Pain? The Pyramid of Pain is a model that ranks types of attacker “indicators” by **how painful it is for the attacker to change them** when defenders detect or block those indicators.

The higher the indicator on the pyramid, the harder it is for the attacker to replace, and the more strategic value it gives to defenders if they detect or disrupt it.

Think of it like this: forcing an attacker to change a **file hash** is like asking them to repaint a car — easy. Forcing them to change their **entire tactics** or tools is like asking them to design a new car from scratch — much harder.

The pyramid (bottom → top)

From easiest for attacker to change (least painful) → to hardest (most painful):

1. **Hashes** (bottom — least painful)
2. **IP Addresses**
3. **Domain Names**
4. **Network/Host Artifacts** (e.g., specific URIs, registry keys, filenames, packet signatures)
5. **Tools** (malware families, frameworks, exploit kits)
6. **TTPs — Tools, Techniques & Procedures** (top — most painful)

Walkthrough with examples and why they matter

1) Hashes (MD5/SHA256)

- **What it is:** A fingerprint of a file (e.g., the hash of a malware binary).
- **Example:** SHA256 = 3a7bd3... for evil.exe.
- **Why low value:** Attackers recompile or pack the binary — hash changes instantly.

- **Defender use:** Short-term block/quarantine and share IOC; useful for malware sample handling and quick containment.

2) IP Addresses

- **What it is:** The numeric address the attacker's server uses for C2 or exfil.
- **Example:** 203.0.113.45 used as a command-and-control server.
- **Why low-medium value:** Easy for attacker to move to another host or use cloud infrastructure. IP blocking helps quickly but is temporary.

3) Domain Names

- **What it is:** Hostnames used for phishing or C2 (e.g., badsite-example[.]com).
- **Example:** A phishing domain or a domain registered to host RAT C2.
- **Why medium value:** Slightly harder than IP (needs new domain registration), but attackers can register new domains or use compromised legitimate domains.

4) Network / Host Artifacts

- **What it is:** Patterns or artifacts like a particular HTTP URI used by a beacon, a registry key used for persistence, or a specific filename/location.
- **Example:** HTTP POST to /api/v1/update every 90s with a specific JSON field; registry run key HKCU\...\Run named svchostHelper.
- **Why higher value:** Changing these might require modifying code/config or operational patterns; attackers need to alter how their tool behaves.

5) Tools (malware families & frameworks)

- **What it is:** The actual malware or offensive toolset (e.g., Emotet, Cobalt Strike, Mimikatz).
- **Example:** A specific RAT binary family or a particular exploit kit.
- **Why high value:** Replacing a tool requires developing or acquiring another tool; often costly and time-consuming.

6) TTPs — Techniques, Tactics & Procedures (top — most painful)

- **What it is:** The attacker's overall way of operating: the sequence of steps, methods, and behaviors used to accomplish goals (e.g., spearphish → macro → PowerShell download → registry persistence → credential dump → use of RDP).
- **Example:** An adversary repeatedly uses living-off-the-land (LOL) tools, PowerShell-based loaders, and scheduled tasks for persistence.
- **Why highest value:** Changing TTPs means the attacker must rethink strategy and workflow — often expensive and slow. Detecting TTPs gives defenders durable protection.

Why defenders should aim higher on the pyramid

- **Short-term controls (bottom)** — blocking hashes, IPs, or domains buys quick wins, but attackers rapidly evolve around them.
- **Long-term wins (top)** — detecting and blocking TTPs forces attackers to change their behavior or tools, increasing their operational cost and improving long-term security for you.

In other words: **stop chasing hashes — hunt for behaviors.**

Practical guidance: move your defenses up the pyramid

Short checklist (actions to shift from low → high)

- Don't rely only on lists of hashes/IPs/domains.
- Implement behavior-based detections (process parenting, execution chains, command-line patterns).
- Use EDR/endpoint sensors to detect techniques (e.g., unusual memory access, LSASS reading).
- Enforce application control (AppLocker/WDAC) to restrict unknown executables.
- Enable rich logging (Windows: Sysmon, PowerShell Script Block Logging, ETW; Network: proxy logs, TLS metadata).
- Use threat intelligence to map observed indicators to **TTPs** (for example, via MITRE ATT&CK).
- Create hunting queries for patterns (e.g., Office → PowerShell encodings, abnormal remote desktop usage).
- Harden environment: MFA, least privilege, patching, removal of local admin rights.

Real-world analogy — a parking-lot example

- Hash = identifying a car by its license plate (easy to change).
- IP = identifying the parking spot number (easy to move to another spot).
- Domain = identifying the owner who rents the space (needs some paperwork change).
- Network/host artifact = the car's custom stereo wiring (requires some work to change).
- Tool = the make & model of the car (harder to replace).
- TTPs = the route, the schedule, and the whole plan of how the driver uses many cars and spots (very hard to change).

If you want to prevent a thief, catching the license plate helps right now — but catching and analyzing the thief's whole routine (TTPs) stops many future thefts.

Example: Applying Pyramid of Pain to a phishing-based RAT attack

- **Observed:** User opens a malicious Word doc → winword.exe spawns a powershell.exe with -EncodedCommand → file saved to C:\Users\...\AppData\temp\svchost.tmp → connects to bad.example.com → uses EvilRAT → creates registry Run key svchostHelper.
- **Indicators by pyramid level:**
 - Hash: svchost.tmp hash (block quickly).
 - IP: IP of bad.example.com (block firewall short-term).
 - Domain: bad.example.com (block and report).
 - Artifacts: command-line pattern winword.exe -> powershell -EncodedCommand and file path AppData\temp\svchost.tmp.
 - Tool: EvilRAT family (detect signatures and behaviors).
 - TTPs: phishing → macro → PowerShell downloader → registry persistence → credential dump (target the sequence).
- **Best defender focus:** build detections for the **command-line parent-child chain**, script block logging that flags encoded PowerShell from Office, and block macros & application whitelisting — these target artifacts/tools/TTPs (higher up) and are longer-lasting.

Quick templates you can use (practical)

- **Detection rule idea:** alert when winword.exe or excel.exe spawns powershell.exe or cmd.exe, or when Office processes write executables to temp folders.
- **Hunting query idea:** search for processes that access LSASS memory or make suspicious registry run key writes.
- **Mitigation idea:** disable macros by policy; enable robust EDR with script logging; restrict outbound traffic to known good; require MFA for administrative accounts.

TL;DR (one-paragraph summary)

The Pyramid of Pain ranks indicators (hashes → IPs → domains → artifacts → tools → TTPs) by how hard it is for attackers to change them. Lower-level indicators are easy for attackers to change and give temporary wins for defenders; focusing on higher-level indicators — especially **TTPs** and **tools** — forces attackers to change their behavior, delivering longer-term defensive value. Build detections and controls that target artifacts, tools, and TTPs rather than only blocking hashes and IPs.